ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

[Thesis Title]

Tomás Domingues

A dissertation submitted in partial fulfillment of the requirements for the degree of Master of Science, Specialisation Area of Cybersecurity And Systems Administration

Advisor: Dr. Jane Smith
Supervisor: Dr. James Smith

Porto, December 11, 2024

Statement of Integrity

I hereby declare having conducted this academic work with integrity.

I have not plagiarised or applied any form of undue use of information or falsification of results along the process leading to its elaboration.

Therefore the work presented in this document is original and authored by me, having not previously been used for any other end.

I further declare that I have fully acknowledged the Code of Ethical Conduct of P.PORTO.

ISEP, Porto, December 11, 2024

Dedicatory

The dedicatory is optional. Below is an example of a humorous dedication.

"To my wife Marganit and my children Ella Rose and Daniel Adam without whom this book would have been completed two years earlier." in "An Introduction To Algebraic Topology" by Joseph J. Rotman.

Abstract

This document explains the main formatting rules to apply to a TMDEI Master Dissertation work for the MSc in Computer Engineering of the Computer Engineering Department (DEI) of the School of Engineering (ISEP) of the Polytechnic of Porto (IPP).

The rules here presented are a set of recommended good practices for formatting the disseration work. Please note that this document does not have definite hard rules, and the discussion of these and other aspects of the development of the work should be discussed with the respective supervisor(s).

This document is based on a previous document prepared by Dr. Fátima Rodrigues (DEI/ISEP).

The abstract should usually not exceed 200 words, or one page. When the work is written in Portuguese, it should have an abstract in English.

Please define up to 6 keywords that better describe your work, in the *THESIS INFORMATION* block of the `main.tex` file.

Keywords: Keyword1, ..., Keyword6

Resumo

Trabalhos escritos em língua Inglesa devem incluir um resumo alargado com cerca de 1000 palavras, ou duas páginas.

Se o trabalho estiver escrito em Português, este resumo deveria ser em língua Inglesa, com cerca de 200 palavras, ou uma página.

Para alterar a língua basta ir às configurações do documento no ficheiro `main.tex` e alterar para a língua desejada ('english' ou 'portuguese')[1]. Isto fará com que os cabeçalhos incluídos no template sejam traduzidos para a respetiva língua.

[1] Alterar a língua requer apagar alguns ficheiros temporários; O target **clean** do **Makefile** incluído pode ser utilizado para este propósito.

Acknowledgement

The optional Acknowledgment goes here... Below is an example of a humorous acknowledgment.

"I'd also like to thank the Van Allen belts for protecting us from the harmful solar wind, and the earth for being just the right distance from the sun for being conducive to life, and for the ability for water atoms to clump so efficiently, for pretty much the same reason. Finally, I'd like to thank every single one of my forebears for surviving long enough in this hostile world to procreate. Without any one of you, this book would not have been possible." in "The Woman Who Died a Lot" by Jasper Fforde.

Contents

List of Figures **xv**

List of Tables **xvii**

List of Algorithms **xix**

List of Source Code **xxi**

List of Acronyms **xxiii**

1 Introduction **1**
1.1 Context . . . 1
1.2 Problem . . . 2
1.3 Objectives . . . 2
1.4 Research Contextualization . . . 3
1.5 Dissertation Structure . . . 3

2 State of the Art **5**
2.1 Theoretical Introduction . . . 5
2.1.1 Security Operations Center . . . 5
Definition and Characteristics of a SOC . . . 6
Key Responsibilities of a SOC . . . 6
Tiers of Operation . . . 6
Triage Specialist . . . 7
2.1.2 Security Information and Event Management . . . 8
Architectural Components . . . 9
2.1.3 Machine Learning . . . 10
2.1.4 Machine Learning Models . . . 12
Decision Trees . . . 12
Ensemble classifiers . . . 13
Random Forests . . . 15
Naive Bayes . . . 17
2.2 Automatic Classification of Security Alerts in SOC . . . 18
2.3 Technologies . . . 21
2.3.1 QRadar . . . 21
2.3.2 Jira . . . 21

3 Method and Implementation **23**
3.1 Method . . . 23

4 Chapter 4 **25**

5 Chapter 5 **27**

6 Results Discussions **29**
6.1 Conclusions 29

Bibliography **31**

List of Figures

2.1 SOC Analyst Tier Responsibilities, from (Kokulu et al. 2019). 7
2.2 SIEM Architecture 10
2.3 A basic structure of a Decision Tree, based on (Chauhan 2022) 12
2.4 Diagram of Parallel Ensemble Learning from (Mienye and Sun 2022) 14
2.5 Diagram of Sequential Ensemble Learning. from (Mienye and Sun 2022) . 15
2.6 Workflow of Random Forests: Bootstrapping, Decision Trees, and Aggregated Predictions from (Yang et al. 2019) 16
2.7 Schematic Diagram of the Naive Bayes Classification Process from (Sneha and Gangil 2019) 17

List of Tables

2.1 Summary of the Existing Related Works based on Ali, Shah, and ElAffendi (2024). 19

List of Algorithms

List of Source Code

List of Acronyms

AI	Artificial Intelligence.
CND	Computer Network Defense.
CSIRT	Computer Security Incident Response Team.
ERD	Endpoint Detection and Response.
IDS	Intrusion and Detection System.
IPS	Intrusion and Prevension System.
ML	Machine Learning.
SIEM	Security Information and Event Management.
SOC	Security Operations Center.

Chapter 1

Introduction

This chapter provides context for automating alert triage and its significance in enhancing security operations in a SOC, particularly for managing high volumes of security alerts. It begins with defining the problem and detailing the challenges associated with manual alert analysis and the critical requirements for automation. First, the objectives of this thesis are presented; second, this project's expected outcomes, including efficiency and accuracy improvements, are outlined, along with the selected approach for integrating an existing ML model to automate alert classification. Finally, an overview of the report's structure is provided.

1.1 Context

In today's world, cybersecurity has become essential for organizations of all sizes. (Vielberth et al. 2020) The rapid advancement of technology, particularly artificial intelligence, has fostered innovation and growth and equipped cybercriminals with increasingly sophisticated tools and techniques. Cyber threats have surged over recent years, and this trend is expected to continue, with global cybersecurity spending projected to rise from $6 trillion to $30 trillion by 2030. (Sovilj et al. 2020)

As these threats become more complex and frequent, Security Operations Center (SOC) have emerged as the primary line of defense. SOCs are crucial in monitoring, detecting, and responding to cyber threats, ensuring the protection of organizations' core digital assets—data, applications, and infrastructure. By combining human expertise with advanced technology, SOCs proactively defend systems, respond to incidents, and work to minimize the impact of potential breaches, safeguarding essential operations. (Jalalvand et al. 2024)

However, managing the overwhelming number of alerts generated daily within SOCs presents a significant challenge. Analysts must review and respond to thousands of tickets, a process that can quickly lead to burnout, as stated by Tines (2023), especially when the majority of alerts are false positives. A study found that many organizations receive over 10,000 alerts daily, with more than 50% being false positives. (CriticalStart 2019)

One of the stages in the SOC workflow with a higher alert volume is the initial triage process. At this stage, Tier 1 analysts are responsible for gathering raw data, assessing alarms, and determining the urgency and nature of each alert. For every ticket, they must evaluate whether the alert is valid or a false positive, assign priority based on the alert's severity, and identify any potential high-risk incidents. (Vielberth et al. 2020) This process demands significant time and effort, placing a heavy mental load on analysts as they deal with large volumes of information.

Integrating automation in the triage process, particularly for determining alert criticality and classification, could substantially reduce analyst fatigue. Machine Learning (ML) techniques are up-and-coming for processing large data volumes. They facilitate the classification and prioritization of alerts, enabling analysts to focus on high-priority threats. (Jalalvand et al. 2024)

This work explores how SOCs can use automation and ML to streamline alert triage, improve SOC efficiency, and more effectively protect organizations from an ever-evolving cyber threat landscape.

1.2 Problem

The main objective of this work is to develop an independent program that automates the triage of security alerts within a Security Operations Center (SOC) environment. This automation aims to enhance the overall efficiency and accuracy of the triage process, reduce the number of false positives, and alleviate the workload during the initial stages of the SOC workflow.

Improving the speed of analyzing initial alerts is crucial in a Security Operations Center (SOC) workflow. Faster analysis enables SOC analysts to respond to incidents quickly, which helps minimize the time that threats remain undetected. Without automation, this initial alert analysis depends heavily on manual inspection. Analysts face the challenge of sifting through a large volume of alerts to identify potential threats, which is time-consuming and susceptible to human error.

This reactive, manual method increases response times and raises the risk of missing critical threats. Analysts must manually classify alerts, assess their severity, and determine appropriate response actions while managing a continuous influx of new alerts. This approach can strain resources and often leads to alert fatigue, making it difficult for analysts to prioritize and respond effectively.

Key metrics such as Mean Time to Detect (MTTD) and Mean Time to Respond (MTTR) can significantly improve by reducing the time required for initial analysis. These metrics are essential for maintaining strong cybersecurity defenses. Automation in alert triage allows for faster and more accurate threat identification, enabling analysts to focus on high-priority threats. This not only increases overall productivity but also reduces fatigue among analysts, while boosting the effectiveness of cybersecurity.

This efficiency results in a more manageable workload, decreased burnout, and greater job satisfaction for employees. For the organization, streamlined response times enhance the security posture, lower the risk of breaches, and ultimately protect critical assets more effectively.

1.3 Objectives

The objectives for this project are as follows:

1. Literature search and review
    - A research into the latest advancements in security analysis and a thorough analysis of the tools, methodologies, and approaches researchers use in SOC automation and machine learning applications.

2. Study of existing Machine Learning (ML) models
   - An analysis of different models for alert triage in SOC environments and related fields, aiming to identify and improve the adopted approach.
3. Collection of security alert datasets
   - A compilation of accessible datasets on security alerts organized to supply training data for the selected ML model.
4. Integration with SIEM and other data sources
   - Connect the SIEM and relevant data sources for real-time data ingestion and analysis.
5. Developing a custom dashboard for analysts to submit feedback on the machine learning model.
   - While the ML model will be trained with existing datasets before deployment, it is still beneficial to provide a method for analysts to continue training the model over time.
6. Testing and evaluation of the selected ML model
   - The selected ML model will be tested with live data, followed by an analysis to assess its effectiveness in categorizing alerts and minimizing false positives.

1.4 Research Contextualization

1.5 Dissertation Structure

[Structure]

Chapter 2

State of the Art

This chapter is divided into three parts. Part I starts on the theoretical side by introducing Machine Learning, Security Operation Centers, Security Information and Event Managers and advanced threat detection and response mechanisms such as Intrusion and Prevension System (IPS), Intrusion and Detection System (IDS) and Endpoint Detection and Response (ERD). It also explains how popular ML classification models work and how to evaluate them. Understanding these topics is the groundwork for material considered later in the chapter.

The second part focuses on current works on the Automatic Classification of Security Alert Tickets. It starts with general information about these systems and then reviews several publicly available datasets. Finally, the chapter ends with a detailed assessment of existing systems similar to the one presented in this thesis. We have separated this analysis from the overview to explore their methods further.

The third and final part discusses existing technologies that SOC may utilize. These technologies are designed to support the workflow outlined in the first section about SOC. At least three different technologies will be presented and discussed for each aspect of the SOC workflow. The section will conclude with a summary of the key differences among the technologies and, if relevant, evaluate which one may be superior.

2.1 Theoretical Introduction

Before diving into the technologies, tools, and methodologies used for automating the classification of security alert tickets, it is essential to understand some fundamental concepts first. Concepts like, SOCs, Security Information and Event Management (SIEM) systems, Artificial Intelligence (AI)—particularly ML—and advanced threat detection and response mechanisms. Such a foundation is indispensable for comprehending these systems' functions, construction work, advantages and limitations, and the actual performance comparison among different methods. The chapter introduces SOC and how it functions diving into , then moves on to the foundations of ML and provides an overview of models frequently used for classification problems. It also provides an overview of the methods for assessing classification model performance to set this in context for how these are used within the project.

2.1.1 Security Operations Center

The complexity and frequency of cyber threats are increasing (Arianna 2024), which has led to the emergence of SOCs as a critical component of modern IT enterprises. SOCs are

the primary defenders in incident response planning, vulnerability management, and regulatory compliance. In today's interconnected world, integrating security operations to reduce defensive barriers allows organizations to optimize resources, enhance security posture, and safeguard critical assets.

A SOC is a unit (Rutledge 2024) that provides tailored and centralized Computer Network Defense (CND) (Zimmerman 2014). It defends computer networks against the growing world of cyber threats. The main objective of a SOC is to ensure continuous monitoring and incident response for enterprise systems (Zimmerman 2014). This primarily focuses on preventing unauthorized access, cyber-attacks, and data breaches.

This 24/7 facility leverages advanced technologies and skilled information security professionals to monitor the network continuously (Zimmerman 2014). With sophisticated tools to detect anomalies, a SOC can address threats before they escalate.

Definition and Characteristics of a SOC

Zimmerman (2014) defined a SOC as:

> "A team primarily composed of security analysts organized to detect, analyze, respond to, report on, and prevent cybersecurity incidents."

This definition integrates elements from various sources, including the historical definition of Computer Security Incident Response Team (CSIRT) as detailed in references (Shirey 2007) and (Brownlee and Guttman 1998).

For an organization to qualify as a SOC, according to Zimmerman (2014), it must:

1. Establish a system for constituents to report cybersecurity incidents.
2. Provide comprehensive support for managing and resolving incidents effectively.
3. Convey incident-related information to internal and external stakeholders.

Key Responsibilities of a SOC

A SOC has several critical missions, as outlined by various sources (Muniz, McIntyre, and AlFardan 2015; Zimmerman 2014):

- Preventing cybersecurity incidents by implementing proactive measures such as vulnerability scanning and threat analysis.
- Monitor, detect, and analyze potential security intrusions.
- Handle confirmed incidents and coordinate resources for effective countermeasures.
- Providing stakeholders with situational awareness regarding cybersecurity incidents, trends, and threats.

Tiers of Operation

Analysts within a SOC operate in tiers (Vielberth et al. 2020). Tier 1 analysts monitor and conduct initial investigations, escalating complex cases to Tier 2 analysts, who perform in-depth analyses and take further actions like blocking activities or deactivating accounts. Generally, higher-tier analysts handle more complex incidents, which require more time to resolve.

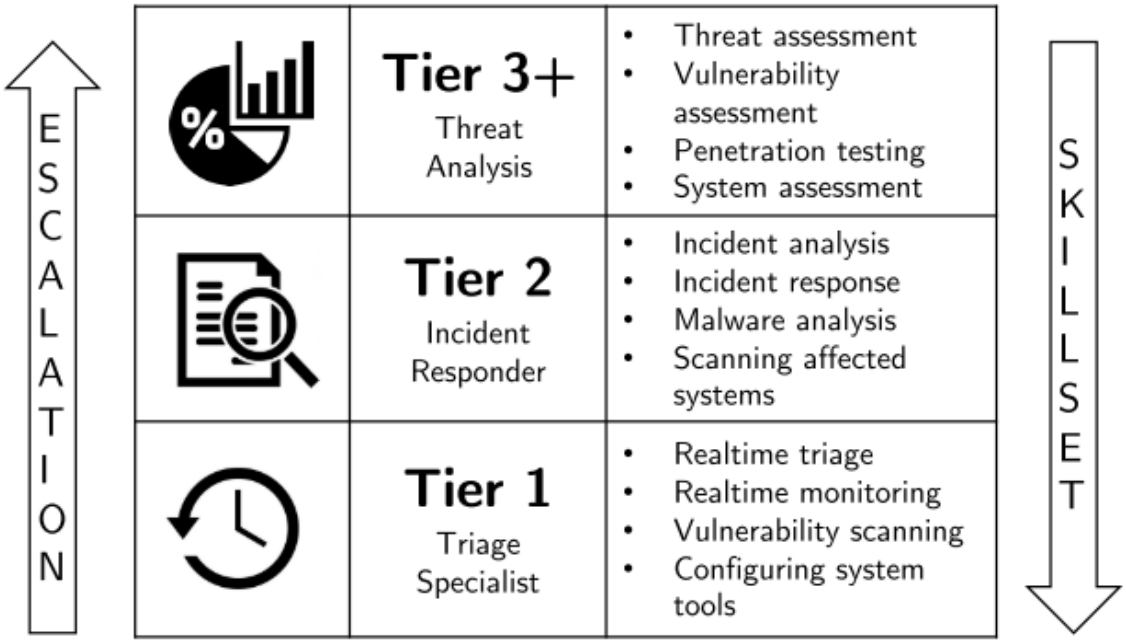

Figure 2.1: SOC Analyst Tier Responsibilities, from (Kokulu et al. 2019).

Extra levels may exist that handle responsibilities like threat hunting, vulnerability assessments, and penetration testing. These levels are collectively termed Tier 3+. Figure 2.1 was taken from a paper on a qualitative study on a SOC (Kokulu et al. 2019) and illustrates a visual representation of these tiers and their associated tasks. Not all SOCs follow a hierarchical model. In some collaborative frameworks, team members may possess comparable skill sets, enabling them to manage incidents independently (Kokulu et al. 2019).

Triage Specialist

Since this project focuses on automating the triage process, it is crucial to gain a detailed understanding of the role of a triage specialist, their functions, and the advantages of automated triage in comparison.

Tier 1 analysts, also known as triage specialists, play a critical role in the initial stages of a SOC workflow (Vielberth et al. 2020).

As stated by Vielberth et al. (2020), a Tier 1 analyst's primary responsibilities include:

1. Collecting and analyzing raw data.
2. Reviewing alarms and alerts generated by monitoring systems.
3. Determining the validity and criticality of each alert.

They must improve alerts with additional contextual information and decide whether an alert represents a real threat or a false positive (Hámornik and Krasznay 2018; Sundaramurthy et al. 2014). This process demands meticulous attention to detail, as the triage specialist must assess individual alerts, notify potential high-risk events, and prioritize them according to their severity (Tao 2018).

The repetitive nature of triage work, coupled with the need to escalate unresolved issues to Tier 2 analysts, can result in mental fatigue and burnout (Iamnitchi et al. 2017; Tines 2023).

This exhaustion affects individual performance and can compromise the overall efficiency of the SOC, as delayed or missed alerts may result in critical threats going undetected (CriticalStart 2019).

In a 2018 study (Crowley and Pescatore 2018), 53% of respondents in a security survey identified inadequate automation as the most common shortcoming.

This study demonstrates that effectively structured and implemented automation can help mitigate some or many of a SOC's weaknesses, particularly in the repetitive aspects of the SOC's workflow, such as the triage process.

2.1.2 Security Information and Event Management

The increasing complexity of cybersecurity threats has compelled organizations to implement advanced technologies to protect their digital assets. SIEM systems have emerged as vital tools in this context (Shaw 2022).

SIEM systems gather and centralize security-related data to detect threats and respond to incidents effectively. These systems connect logs from various sources to support security analytics, enabling real-time monitoring and retrospective analysis of past events (Shaw 2022). SIEM systems normalize raw log data, correlate events, generate reports and issue alerts, and automate responses to security incidents using enriched contextual data (Vielberth 2021). They integrate with cyber threat intelligence platforms, providing human analysts with advanced visual tools for seamless information sharing between organizations. Additionally, they retain event data over extended periods, ensuring robust log management capabilities.

The key features of a SIEM system, as gathered from various published sources (Ali, Shah, and ElAffendi 2024; Harper et al. 2010; Sheeraz et al. 2023), are:

- **Log Collection:** SIEM gathers log data from various network devices such as servers, firewalls, and switches. Data can be collected using two methods:
  1. **Agent-based collection:** An intermediary agent collects and forwards logs.
  2. **Agent-less collection:** Servers retrieve logs directly from the source devices.
- **Log Aggregation:** Collected logs are analyzed and structured for meaningful insights. Aggregation methods include:
  1. **Push method:** Devices actively send logs to the SIEM.
  2. **Pull method:** SIEM retrieves logs as needed.
- **Parsing and Normalization:** Parsing converts raw logs into structured data, while normalization standardizes logs from diverse sources to eliminate redundancy.
- **Threat Analysis and Detection:** By correlating log data with known threat indicators, SIEM systems identify malicious activities. Statistical methods and predefined rules enhance their ability to detect sophisticated threats.
- **Response Automation:** SIEM systems issue real-time alerts and notifications, enabling rapid responses to potential incidents.
- **Reporting and Visualization:** Advanced reporting tools provide security analysts with actionable insights, enabling detailed investigations and trend analysis.

Architectural Components

The architectural components of a security information and Event Management (SIEM) system consist of several essential elements that enable effective security monitoring, incident detection, and response (Sheeraz et al. 2023).

Data sources provide the raw material for analysis and threat detection (Ali, Shah, and ElAffendi 2024). These include a wide variety of log-generating devices and applications:

- **Network devices**: Firewalls, routers, and switches.
- **Endpoint devices**: Workstations, servers, and mobile devices.
- **Applications**: Web servers, databases, and cloud platforms.

A variety of data sources is crucial for effective monitoring and threat detection.

Data collection is a vital step with two main approaches to consider:

- **Agent-based collection**: Agent-based collection uses proxy agents on endpoint devices for better control and flexibility in log collection, but it is costly and complex to manage.
- **Agent-less collection**: Agent-less collection allows devices to send data directly to the SIEM, simplifying deployment but may reduce efficiency in high-volume data environments.

The **SIEM processing engine** is one of the critical components (Sheeraz et al. 2023) responsible for:

- **Parsing**: The conversion of raw log data into a structured format used for analysis.
- **Normalization**: Ensure the log formats are standardized to facilitate easier comparisons.
- **Correlation**: Finding relationships between events to strengthen security posture or incident response.

Storage and rationalization are vital for ensuring scalability and compliance. SIEM systems must store logs for future analysis. This means there has to be enough storage and a logical way of organizing this data for scalability or compliance requirements like GDPR and HIPAA (Sheeraz et al. 2023). Effective storage solutions should scale dynamically to handle large datasets without compromising performance or compliance standards.

Visualization and reporting tools play a key role in making data accessible and actionable. Critical functions that significantly benefit from good data visualization and reporting tools are incident investigation, trend analysis, and compliance audits (Sheeraz et al. 2023). These features not only improve user experience but also aid in making data actionable and accessible. Organizations can use these tools to conduct incident investigations, identify patterns occurring over time, and monitor compliance (Sheeraz et al. 2023). This comprehensive approach enables the development of scalable systems that can effectively handle increasing data demands.

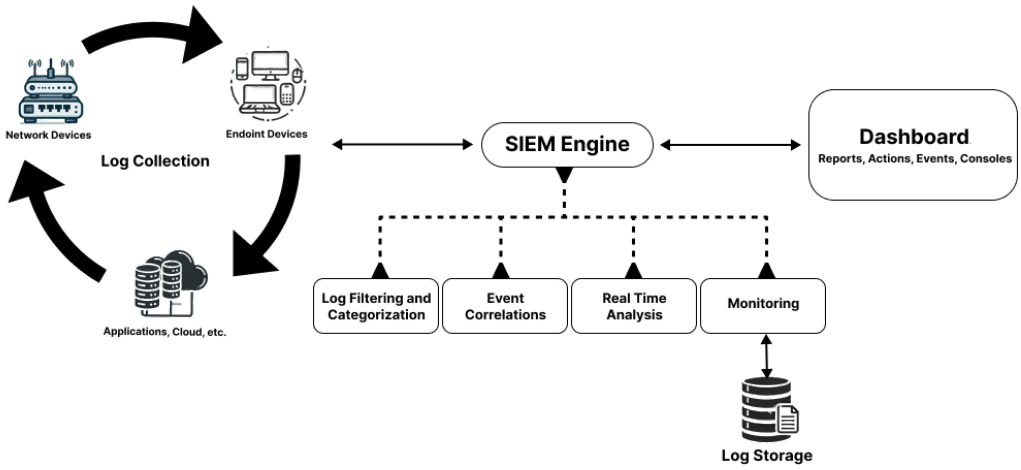

Figure 2.2: SIEM Architecture

Figure 2.2 illustrates the architecture of a SIEM system, highlighting its critical components and their interactions. Data flows from network and endpoint devices, as well as applications, to the log collection module. The SIEM processing engine then parses, normalizes, and correlates this data for real-time security alert analysis, event monitoring, and threat detection.

The diagram also emphasizes the importance of visualization and reporting tools for turning data into actionable insights.

The graphic illustrates how these elements integrate into a SIEM.

2.1.3 Machine Learning

ML is a core part of AI but also overlaps with data mining, statistics, probability, and mathematics (Mohri, Rostamizadeh, and Talwalkar 2012). Unlike traditional rule-based systems that rely on predefined logic, ML uses induction—it learns patterns from past data and forms assumptions that can be generalized to new cases (Ali, Shah, and ElAffendi 2024). This method relies on datasets, which are groups of examples the ML algorithm analyzes to find patterns (Mohri, Rostamizadeh, and Talwalkar 2012; Suthaharan 2016). The goal is to use these learned patterns to predict or describe new data.

There are three primary types of techniques employed in machine learning, (Mohri, Rostamizadeh, and Talwalkar 2012):

1. **Reinforcement Learning**: This is a subset of machine learning in which an agent learns by interacting with the environment (Moradi, Acker, and Denil 2023). It observes the environment, selects an action, and receives a reward if the action is beneficial or a penalty if it is detrimental. Over time, it refines its approach to achieve maximum rewards.

2. **Supervised Learning**: In this type of machine learning, models are trained on labeled data, meaning that each example includes input features and the corresponding expected output (or label). The model learns to map the inputs to the outputs, enabling it to predict the output for new, unseen data.

3. **Unsupervised Learning**: Unsupervised learning analyzes unlabeled data to reveal patterns without predefined labels. Unlike supervised learning, it allows algorithms to explore data independently. It is often used for clustering similar data points or modeling probability distributions. This approach is valuable for understanding the inherent organization in data without prior knowledge.

Supervised learning can be further categorized into several types, (Mohri, Rostamizadeh, and Talwalkar 2012):

- **Regression**: Regression algorithms predict numerical values within a continuous range by analyzing input data to identify patterns. They can forecast future values, such as calculating the next number in a sequence based on previous numbers and trends. Techniques like linear regression, polynomial regression, and others enable these algorithms to draw conclusions and make predictions effectively.

- **Similarity**: Similarity algorithms analyze and compare two distinct instances to measure their resemblance. They are vital in recommender systems for suggesting products based on user preferences and in visual identity tracking and verification by comparing images or features. Their versatility makes them essential in data analysis, security, and personalized user experiences.

- **Classification**: Classification algorithms classify the input data into predefined groups. Classification tasks can be binary when there are only two possible categories (such as a yes-no decision) or multiclass when the number of categories exceeds two, such as recognizing handwritten letters in the alphabet.

However, ML has its limitations. Since datasets are finite, no algorithm can predict every scenario, which highlights an essential aspect of inductive reasoning: it can suggest likely outcomes but cannot ensure certainty (Mohri, Rostamizadeh, and Talwalkar 2012; Suthaharan 2016).

In ML, achieving an optimal model involves balancing between two critical concepts: **bias** and **variance**. Bias refers to the error introduced when a model is excessively simplistic, which can lead to underfitting. Underfitting occurs when the model fails to capture the underlying patterns of the data, resulting in poor performance on both training and unseen datasets (ElSahly and Abdelfatah 2023). On the other hand, variance arises when a model becomes overly complex and sensitive to the fluctuations in the training data, culminating in overfitting. Overfitting means the model performs exceptionally well on the training dataset but poorly on new, unseen data because it has memorized the noise instead of learning the actual signal (ElSahly and Abdelfatah 2023).

The primary objective in constructing a machine learning model is to identify the appropriate level of complexity with the right balance, ensuring the model generalizes well and performs effectively on new data (Suthaharan 2016).

Despite these challenges, ML continues to evolve, with increasingly sophisticated algorithms enabling breakthroughs in many fields (Mohri, Rostamizadeh, and Talwalkar 2012).

2.1.4 Machine Learning Models

The processes and computations of training a machine learning model are structured within a framework. This chapter will briefly overview the most frequently used algorithms relevant to this project's goals.

Decision Trees

Decision Trees (DTs) are a fundamental supervised machine learning algorithm for classification and regression tasks (Huang 2024). They work by splitting data into subsets based on the value of input features, forming a tree-like structure composed of nodes and branches.

The process begins at the root node, representing the entire dataset, and iteratively divides the data into homogeneous subsets using decision nodes until a terminal leaf node is reached, representing the output prediction or class (Chauhan 2022), as seen in Figure 2.3.

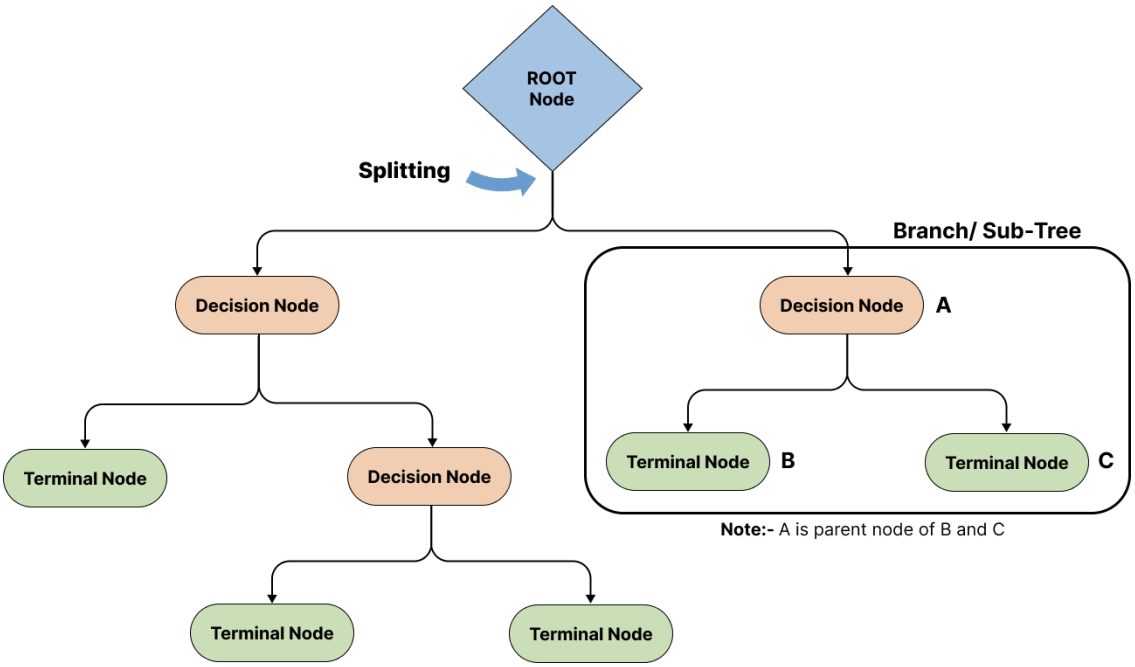

Figure 2.3: A basic structure of a Decision Tree, based on (Chauhan 2022)

The algorithm operates by evaluating all potential splits and selecting the one that optimizes a specific criterion, such as information gain or Gini impurity. Information gain measures the reduction in entropy after a split, while Gini impurity quantifies the likelihood of misclassification at a node.

The entropy of S is defined as:

$$H(S) = \sum_{i=1}^{n} -P(s_i) \times \log_b P(s_i)$$

where S is a set of values $s_1, s_2, \ldots, s_n$, $P(s_i)$ is the probability of observing a certain value, and b is the logarithm base, most commonly 2, e, or 10.

Using entropy, the **information gain (IG)** for a node t and a candidate split x is calculated as:

$$IG(t, x) = H(t) - H(x, t)$$

In contrast, the **Gini index**, another commonly used metric for evaluating splits, is defined as:

$$\text{Gini} = 1 - \sum_{i=1}^{n} P(s_i).$$

At each decision node, the algorithm tests a single feature and branches according to its value, guiding data instances down the tree until they reach a leaf node (Chauhan 2022). During training, the algorithm continues splitting until a stopping criterion is met, such as achieving a maximum tree depth, minimum node size, or no further improvement in the splitting metric (Chauhan 2022).

Pruning techniques prevent overfitting, which occurs when a tree becomes too complex and overly specific to the training data. These involve removing unnecessary branches or nodes to simplify the tree while maintaining predictive accuracy. Pruning can be preemptive (stopping tree growth early) or post hoc (removing branches after the tree is fully grown). This ensures that the decision tree generalizes well to unseen data (Huang 2024).

Decision trees are non-parametric, meaning they do not assume any specific distribution for the data, and they can capture both linear and non-linear relationships (Huang 2024). However, they are sensitive to slight variations in the dataset (Huang 2024), which may cause significant changes in the tree structure. Despite this, they remain popular for their simplicity, interpretability, and ability to handle numerical and categorical data (Chauhan 2022).

Ensemble classifiers

Ensemble learning is a powerful machine learning technique that combines the predictions of multiple base models to achieve superior performance compared to individual learners (Joseph 2022).

The fundamental idea is to address the limitations of single models, such as high variance, high bias, or low accuracy, by leveraging the diversity and complementary strengths of multiple models (Dasarathy and Sheela 1979; Hansen and Salamon 1990; Mienye and Sun 2022).

The concept of ensemble learning has evolved significantly since its inception. Early work by Dasarathy and Sheela (1979) introduced the partitioning of feature spaces using multiple classifiers. Later, Hansen and Salamon (1990) demonstrated that ensembles of artificial neural networks achieved superior predictive performance compared to single networks. Schapire (1990) groundbreaking work laid the foundation for boosting, one of the primary techniques in ensemble learning.

Ensemble learning methods are evaluated on two main principles: **accuracy** and **diversity** of the base learners (Hansen and Salamon 1990; Mienye and Sun 2022).

- **Accuracy**: Each base learner should perform better than random guessing on the given task (Li, Yu, and Zhou 2012).

- **Diversity**: The errors made by individual learners should be uncorrelated, which can be achieved through techniques like subsampling, feature randomization, or algorithmic diversity (Breiman 1996; Schapire 1990).

A model is accurate if it generalizes well on unseen instances, while diversity ensures that the errors of individual base models are not correlated. Achieving this balance is crucial for effective ensembles.

Ensemble Classifiers offer several advantages over single models, making them a popular choice in diverse applications of machine learning:

- **Improved Generalization**: Ensembles reduce overfitting and improve generalization by aggregating the predictions of multiple learners (Hansen and Salamon 1990).
- **Bias-Variance Trade-off**: Techniques like bagging reduce variance while boosting minimizes bias, addressing the critical limitations of individual learners (Li, Yu, and Zhou 2012; Mienye and Sun 2022).
- **Adaptability**: Ensembles can be designed to work with homogeneous (same algorithm) or heterogeneous (different algorithms) base learners, making them versatile across domains (Mienye and Sun 2022).

Ensemble learning methods are broadly categorized into:

- **Parallel Ensembles**: Base learners are trained independently on different subsets of data or features. Techniques like bagging and Random Forests fall into this category (Breiman 1996).
- **Sequential Ensembles**: Models are trained iteratively, with each learner focusing on correcting the errors of its predecessor. Boosting is the most prominent example of this approach (Schapire 1990).
- **Stacked Ensembles**: Predictions from multiple base models are combined using a meta-model trained on the outputs of the base learners (Mienye and Sun 2022).

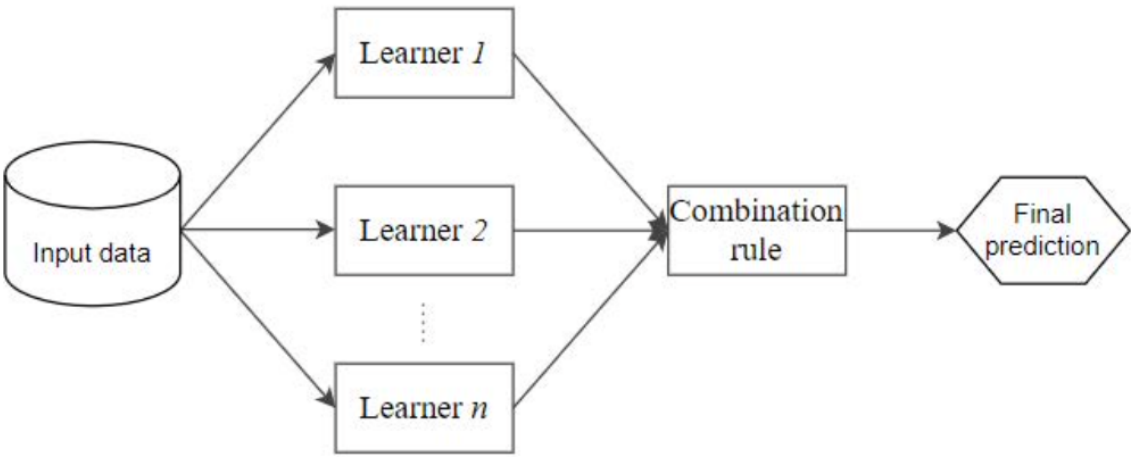

Figure 2.4: Diagram of Parallel Ensemble Learning from (Mienye and Sun 2022)

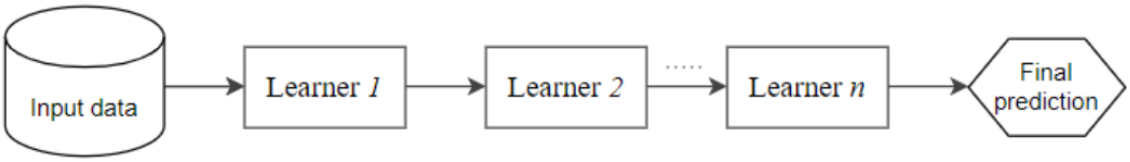

Figure 2.5: Diagram of Sequential Ensemble Learning. from (Mienye and Sun 2022)

Ensemble methods can then be categorized into three main approaches based on how they train and combine base models: **Bagging**, **Boosting**, and **Stacking**. Each method has a distinct mechanism for generating diversity and aggregating predictions, addressing the specific limitations of single models.

Bagging Bagging, which stands for bootstrap aggregating, is a parallel ensemble technique where multiple base models are independently trained on random subsets of the training data, referred to as bootstrapped samples. The final predictions are made by aggregating the outputs of these models, often using majority voting or averaging. Random Forests are a well-known implementation of bagging, recognized for their ability to reduce variance without increasing bias (Mienye and Sun 2022).

Boosting Boosting is a sequential ensemble method that aims to reduce bias by iteratively training base models. Each subsequent model focuses on correcting the errors made by the previous one, assigning higher weights to misclassified samples. Algorithms such as AdaBoost and Gradient Boosting exemplify this approach, often achieving superior performance on complex datasets, but this can lead to increased susceptibility to overfitting (Mienye and Sun 2022).

Stacking Stacking integrates predictions from multiple base models using a meta-model that learns the optimal way to combine their outputs. Unlike bagging and boosting, stacking allows for heterogeneous base learners, providing greater flexibility and diversity. The meta-model is typically trained on the predictions made by the base models, enabling it to make more accurate decisions (Mienye and Sun 2022).

Random Forests

Random Forests represent a significant advancement in machine learning, particularly in the domain of ensemble classifiers (Ali, Shah, and ElAffendi 2024).

As a bagging-based ensemble method, Random Forests combine multiple decision trees to improve classification accuracy and robustness. This approach leverages the strengths of individual trees while mitigating their limitations, such as overfitting, by aggregating their predictions (Ali, Shah, and ElAffendi 2024).

The Random Forest algorithm begins by generating multiple decision trees, each trained on a random subset of the training data through bootstrapping[1]. Additionally, features are

[1]Bootstrapping is a resampling technique where random samples are drawn with replacement from the dataset.

randomly sampled at each split to ensure diversity among the trees, a process that reduces the correlation between individual tree predictions (Farooq and Otaibi 2018). Once trained, the predictions of all trees are aggregated, typically through majority voting for classification tasks or averaging for regression tasks, to produce the final output (Nila, Apostol, and Patriciu 2020).

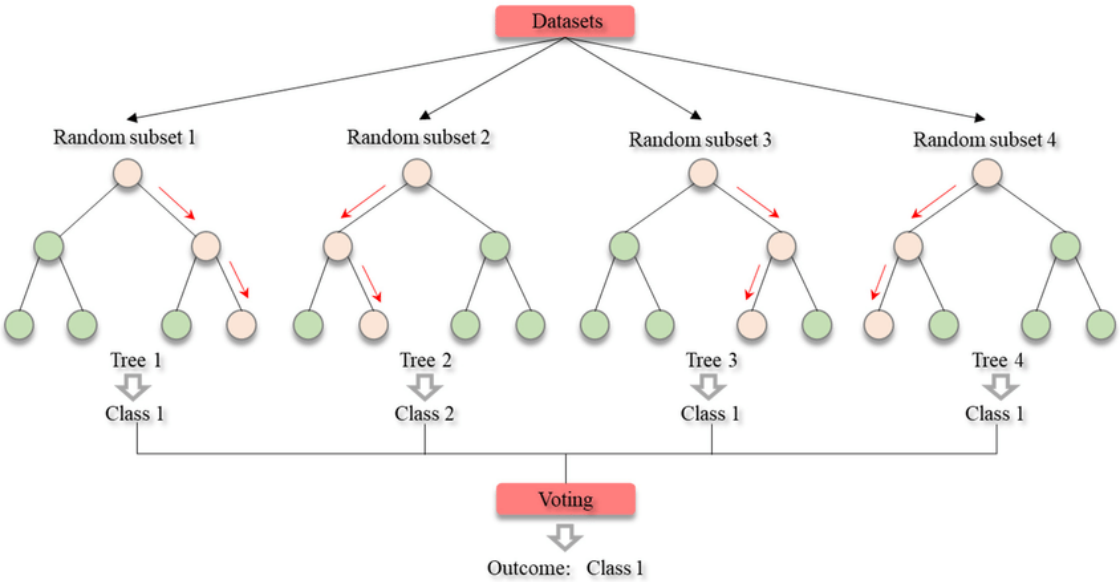

Figure 2.6: Workflow of Random Forests: Bootstrapping, Decision Trees, and Aggregated Predictions from (Yang et al. 2019)

Random Forests are highly valued for their versatility and robustness. They perform exceptionally well with large and high-dimensional datasets, effectively handling numerical and categorical data. Furthermore, their inherent ability to measure feature importance makes them interpretable, a critical requirement in security domains where trust and explanation are essential (Ali, Shah, and ElAffendi 2024).

Random Forests also excel at reducing overfitting, a common issue with individual decision trees. By averaging the predictions of multiple trees, the model achieves better generalization, even when faced with noisy data (Sopan et al. 2019).

Random Forests have been widely adopted in cybersecurity applications, particularly for detecting anomalies, classifying alerts, and predicting the likelihood of malicious behavior. For example, they are effectively used in SIEM systems to process large volumes of log data, identify patterns, and reduce false positives (Ali, Shah, and ElAffendi 2024; Farooq and Otaibi 2018).

Studies have shown that Random Forest-based models can achieve high accuracy in detecting advanced persistent threats (APTs) and other cyberattacks by leveraging their ability to handle complex feature interactions and high-dimensional data (Ali, Shah, and ElAffendi 2024). For instance, combining Random Forests with additional modules for feature selection and preprocessing has improved detection rates and reduced manual intervention in incident response workflows (Nila, Apostol, and Patriciu 2020).

Naive Bayes

The Naive Bayes algorithm is a probabilistic classifier based on Bayes' theorem, which assumes conditional independence among features given the class (Chandra, Challa, and Pasupuleti 2016). This simplicity makes it computationally efficient and easy to implement.

Naive Bayes is particularly effective in scenarios where the assumption of feature independence is approximately attained, such as spam detection and document classification. The model's reliance on this assumption allows it to scale effectively to high-dimensional datasets while remaining computationally efficient (Chandra, Challa, and Pasupuleti 2016).

Naive Bayes calculates the posterior probability of each class given the observed data using the formula:

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

where:

- $P(C|X)$ is the posterior probability of class C given the feature vector X,
- $P(X|C)$ is the likelihood of observing X given class C,
- $P(C)$ is the prior probability of class C,
- $P(X)$ is the probability of the feature vector X.

In practice, the Naive Bayes model estimates the prior probability $P(C)$ for each class and the conditional probability $P(X_i|C)$ for each feature X_i given the class. These probabilities are typically derived from the frequency of occurrences in the training data. For continuous features, it is common to assume a Gaussian distribution and compute the likelihood accordingly. The model assigns a new instance to the class with the highest posterior probability $P(C|X)$ (Chandra, Challa, and Pasupuleti 2016; Nila, Apostol, and Patriciu 2020).

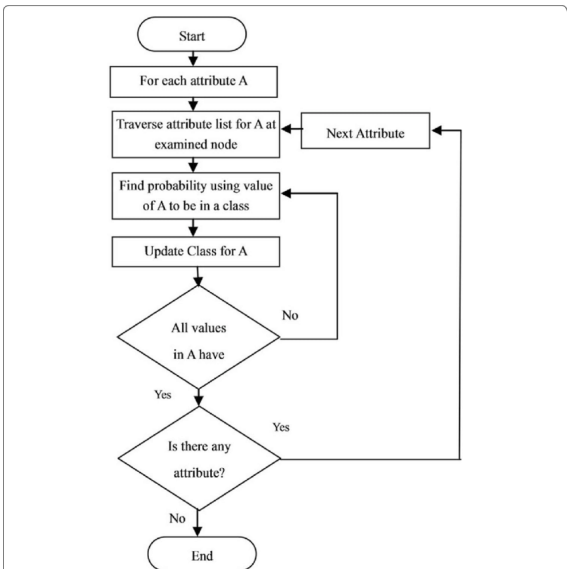

Figure 2.7: Schematic Diagram of the Naive Bayes Classification Process from (Sneha and Gangil 2019)

This decision rule is computationally efficient, making Naive Bayes suitable for real-time prediction tasks (Chandra, Challa, and Pasupuleti 2016; Nila, Apostol, and Patriciu 2020).

However, the strong independence assumption may only hold in some scenarios, potentially leading to suboptimal performance when features are highly correlated. Despite this limitation, Naive Bayes remains popular due to its simplicity and effectiveness in many practical applications (Chandra, Challa, and Pasupuleti 2016).

2.2 Automatic Classification of Security Alerts in SOC

The growing complexity of cyber threats, especially Advanced Persistent Threats (APT), has led to significant advancements in automated detection and mitigation mechanisms. Many of these mechanisms utilize ML techniques. This section examines key works, focusing on their methodologies, limitations, and relevance to the problem of alert triage automation in SOC.

Recent research has explored hybrid approaches to anomaly detection. Saini et al. (2023) proposed a hybrid ensemble model combining Random Forest and XGBoost classifiers, achieving a 99.91% accuracy on the CIC-IDS2017[2] dataset with a False Positive Rate (FPR) as low as 0.12%. Despite its high performance, the model relies on static datasets, which limits its adaptability to evolving attack patterns. Similarly, Ghafir et al. (2018) developed Machine Learning for Advanced Persistent Threats (MLAPT), a multi-phase system incorporating correlation frameworks for APT detection, achieving an accuracy of 84.8%. While this approach improved early-stage prediction, its accuracy was limited compared to ensemble methods and lacked scalability for dynamic environments.

Ali, Shah, and ElAffendi (2024) extended APT datasets to include a diverse range of alerts. They integrated Random Forest and XGBoost models into a SIEM environment, demonstrating the ability to reduce FPR and achieve a near-perfect 99.6% accuracy. However, the computational demands of such systems may hinder practical deployment in resource-constrained environments. On the contrary, Brogi and Tong (2016) utilized Information Flow Tracking (IFT) for APT detection, concentrating on the correlation between the various stages of an attack. While this method excels at tracing complex multi-stage attacks, it requires further automation to support large-scale SOC operations effectively.

Incorporating reinforcement learning, Sethi et al. (2020) introduced a Deep Q-Network (DQN)-based context-adaptive intrusion detection system, which improved FPR and demonstrated robustness against several attacks. Nevertheless, the distributed nature of their approach introduces practical deployment challenges, especially in SOC environments where centralized oversight is critical. Similarly, Nila, Apostol, and Patriciu (2020) explored ML-based triage systems to automate alert classification and reduce false positives. Their work effectively alleviated analyst fatigue by prioritizing actionable alerts, yet the scope of implementation remains limited to isolated systems.

Chandra, Challa, and Pasupuleti (2016) conducted a comprehensive study on email-based APT entry points, explicitly exploring the effectiveness of Bayesian spam filters[3]. These filters demonstrated a strong capability to identify and differentiate between spear-phishing attempts and general spam emails, which is crucial for enhancing cybersecurity measures.

[2] Labeled dataset of network traffic, including normal behavior and various attacks, used to evaluate intrusion detection models.

[3] Bayesian spam filters classify emails as spam or legitimate using probabilities based on Bayes theorem.

However, the authors noted that while their solution provided significant benefits in detecting these specific types of email threats, it needed to address the broader range of APT life cycles. This limitation restricts its effectiveness in SOC environments with diverse attack vectors, requiring a more holistic approach to threat detection and response strategies to encompass the full spectrum of APT strategies and tactics.

Table 2.1: Summary of the Existing Related Works based on Ali, Shah, and ElAffendi (2024).

References	APT Life Cycle Coverage	AI Models Used	Lowest FPR	Highest Accuracy	Limitations
Ghafir et al. (2018)	Complete	Decision Tree, SVM, KNN, Ensemble Classifier	4.5%	84.8%	The blacklist-based detection modules require continuous update. Moreover, the accuracy is lower.
Brogi and Tong (2016)	Complete	No ML model used	High	Not calculated	Information Flow Tracking is used to detect APTs; however, the FPR is high.
Giura and Wang (2012)	Complete	No ML model used	27.88%	Not calculated	Sufficient knowledge is required to set up the mechanism.
Sopan et al. (2019)	No	Random Forest	Not calculated	98.5%	The post-alert decision is not automated; it involves the intervention of security experts.
Farooq and Otaibi (2018)	Partial	SVM, Random Forest	Not calculated	Not calculated	The process anomaly detection is presented formally using One-Class SVM. However, the paper lacks ML-based experimental results.
Sethi et al. (2020)	No	Deep Q-network	0.35%	96.12%	The proposed model enables the detection of APTs.
Nila, Apostol, and Patriciu (2020)	No	ZeroR, OneR, NaiveBayes, SVM, J48, RandomForest	Not calculated	95.5%	The authors did not discuss the FPR of the proposed model.
Chandra, Challa, and Pasupuleti (2016)	Partial	NaiveBayes	Not calculated	87%	The authors did not discuss the FPR of the proposed model.
Saini et al. (2023)	Partial	Random Forest, XGBoost	0.12%	99.91%	This study used existing datasets, posing a challenge because these datasets might not contain the latest attack scenarios.
Ali, Shah, and ElAffendi (2024)	Complete	Random Forest, XGBoost	Not calculated%	99.6%	High resource demand for training; dataset generated in controlled environments, limiting real-world adaptability.

This study introduces a new approach to automating alert triage in SOCs by combining ML models with existing SIEM systems. In contrast with the previous studies, this research will use live data as its training data and will be deployed in a real-world environment. Therefore,

analysts can focus on high-risk issues by analyzing the alerts based on the severity of the cyber threat shown by the ML model.

While the related works reviewed in this study showcase significant advancements in SOC automation and alert triage, none closely reflects this research's objectives. However, beyond academic research, commercially available solutions like Check Point's SOC automation tools demonstrate functionalities that align with aspects of this study. These solutions offer valuable insights into the application of AI and automation in SOC environments, providing a practical point of comparison for the approach proposed in this research.

Check Point's approach to SOC automation stands out for its focus on streamlining SOC operations using artificial intelligence (AI) and automation tools (CheckPoint n.d.).

SOC automation, as described by Check Point, employs AI to take over repetitive and manual tasks in the SOC, such as alert triage, incident response, and threat detection. Their solution includes tools like:

- **Generative AI:** Used to improve usability and streamline workflows, enabling analysts to query and interact with data using natural language.
- **Playbooks:** Predefined and customizable workflows for incident response that enable rapid and consistent remediation.
- **Threat Detection and Analytics:** Leveraging advanced analytics, behavioral insights, and proprietary threat intelligence from Check Point Research and ThreatCloud AI to identify threats and reduce false positives.
- **Malware Analysis and Phishing Detection:** Using sandboxes and natural language processing (NLP) to analyze suspicious files and emails.
- **Infinity Extended Prevention and Response (XDR/XPR):** A platform that integrates automated playbooks with real-time threat intelligence and analytics to correlate events across the security estate, detect sophisticated attacks, and prevent lateral movement.

The research in this study aligns closely with Check Point's SOC automation approach. Both aim to automate alert triage and streamline SOC operations using AI to enhance efficiency. This study also shares key goals, such as reducing false positives, prioritizing alerts, and providing analysts with tools to focus on high-risk threats. Like Check Point, this research emphasizes the use of playbooks for consistent responses and aims to alleviate the burden of repetitive tasks on analysts.

While the Check Point solution is comprehensive and includes a full suite of tools, this research focuses on a customized solution tailored to the specific needs of the organization conducting this study differing in:

1. **Integration with Existing Infrastructure:** This study develops a solution that seamlessly integrates with the organization's current SIEM system, avoiding the need for a complete overhaul of existing systems, as would be required to adopt Check Point's platform.
2. **Live Data and Real-World Deployment:** Unlike Check Point's reliance on proprietary threat intelligence and prebuilt systems, this research involves using live data collected within the organization's environment, ensuring the model is trained and deployed in a real-world context.

2.3 Technologies

2.3.1 QRadar

2.3.2 Jira

Chapter 3

Method and Implementation

3.1 Method

Chapter 4

Chapter 4

Chapter 5

Chapter 5

Chapter 6

Results Discussions

6.1 Conclusions

Bibliography

Ali, Gauhar, Sajid Shah, and Mohammed ElAffendi (Sept. 2024). *Enhancing Cybersecurity Incident Response: AI-Driven Optimization for Strengthened Advance Persistence Threat Detection*. doi: 10.20944/preprints202409.1725.v1. url: https://www.preprints.org/manuscript/202409.1725/v1.

Arianna, By (2024). *Cybersecurity Management Services: Why ITSM Services Essential?* url: https://forlicoupon.it/2024/07/17/cybersecurity-management-services-why-itsm-services-essential/.

Breiman, Leo (1996). "Bagging predictors". In: *Machine Learning* 24 (2), pp. 123–140. issn: 08856125. doi: 10.1007/BF00058655/METRICS. url: https://link.springer.com/article/10.1007/BF00058655.

Brogi, Guillaume and Valérie Viet Triem Tong (Dec. 2016). "TerminAPTor: Highlighting advanced persistent threats through information flow tracking". In: *2016 8th IFIP International Conference on New Technologies, Mobility and Security, NTMS 2016*. doi: 10.1109/NTMS.2016.7792480.

Brownlee, N. and E. Guttman (June 1998). "Request for Comments 2350 Expectations for Computer Security Incident Response," in: url: http://www.ietf.org/rfc/rfc2350.txt.

Chandra, J. Vijaya, Narasimham Challa, and Sai Kiran Pasupuleti (Aug. 2016). "A practical approach to E-mail spam filters to protect data from advanced persistent threat". In: doi: 10.1109/ICCPCT.2016.7530239.

Chauhan, Nagesh (Feb. 2022). *Decision Tree Algorithm, Explained - KDnuggets*. url: https://www.kdnuggets.com/2020/01/decision-tree-algorithm-explained.html.

CheckPoint (n.d.). *What is SOC Automation? - Check Point Software*. url: https://www.checkpoint.com/cyber-hub/threat-prevention/what-is-soc/what-is-soc-automation/.

CriticalStart (2019). *THE IMPACT OF SECURITY ALERT OVERLOAD*. url: https://www.criticalstart.com/wp-content/uploads/2021/02/CS_Report-The-Impact-of-Security-Alert-Overload.pdf.

Crowley, Christopher and John Pescatore (2018). *The Definition of SOC-cess? A SANS Survey*.

Dasarathy, Belur V. and Belur V. Sheela (1979). "A composite classifier system design: Concepts and methodology". In: *Proceedings of the IEEE* 67 (5), pp. 708–713. issn: 0018-9219. doi: 10.1109/PROC.1979.11321. url: https://www.academia.edu/30910041/A_composite_classifier_system_design_concepts_and_methodology.

ElSahly, Osama and Akmal Abdelfatah (Aug. 2023). "An Incident Detection Model Using Random Forest Classifier". In: *Smart Cities* 6 (4), pp. 1786–1813. issn: 26246511. doi: 10.3390/SMARTCITIES6040083.

Farooq, Hafiz M. and Naif M. Otaibi (Dec. 2018). "Optimal machine learning algorithms for cyber threat detection". In: *Proceedings - 2018 UKSim-AMSS 20th International Conference on Modelling and Simulation, UKSim 2018*, pp. 32–37. doi: 10.1109/UKSIM.2018.00018.

Ghafir, Ibrahim et al. (July 2018). "Detection of advanced persistent threat using machine-learning correlation analysis". In: *Future Generation Computer Systems* 89, pp. 349–359. issn: 0167739X. doi: 10.1016/J.FUTURE.2018.06.055. url: https://bradscholars.brad.ac.uk/handle/10454/17614.

Giura, Paul and Wei Wang (2012). "A context-based detection framework for advanced persistent threats". In: *Proceedings of the 2012 ASE International Conference on Cyber Security, CyberSecurity 2012*, pp. 69–74. doi: 10.1109/CYBERSECURITY.2012.16.

Hámornik, Balázs Péter and Csaba Krasznay (2018). "A team-level perspective of human factors in cyber security: Security operations centers". In: *Advances in Intelligent Systems and Computing* 593, pp. 224–236. issn: 21945357. doi: 10.1007/978-3-319-60585-2_21. url: https://www.researchgate.net/publication/318177610_A_Team-Level_Perspective_of_Human_Factors_in_Cyber_Security_Security_Operations_Centers.

Hansen, Lars Kai and Peter Salamon (1990). "Neural Network Ensembles". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 12 (10), pp. 993–1001. issn: 01628828. doi: 10.1109/34.58871. url: https://www.researchgate.net/publication/3191841_Neural_Network_Ensembles.

Harper, Allen et al. (Dec. 2010). *Security Information And Event Management (SIEM) Implementation*. Ed. by MCGRAW-HILL EDUCATION - EUROPE, p. 464. isbn: 9780071701099.

Huang, Xiang (July 2024). "Predictive Models: Regression, Decision Trees, and Clustering". In: *Applied and Computational Engineering* 79 (1), pp. 124–133. issn: 2755-273X. doi: 10.54254/2755-2721/79/20241551. url: https://www.ewadirect.com/proceedings/ace/article/view/13989.

Iamnitchi, Adriana et al. (2017). "An Anthropological Study of Security Operations Centers to Improve Operational Efficiency". In: url: https://digitalcommons.usf.edu/etd.

Jalalvand, Fatemeh et al. (Nov. 2024). "Alert Prioritisation in Security Operations Centres: A Systematic Survey on Criteria and Methods". In: *ACM Computing Surveys* 57 (2), pp. 1–36. issn: 0360-0300. doi: 10.1145/3695462. url: https://dl.acm.org/doi/10.1145/3695462.

Joseph (Aug. 2022). *TensorFlow Ensemble Learning: What You Need to Know - reason.town*. url: https://reason.town/tensorflow-ensemble-learning/.

Kokulu, Faris Bugra et al. (Nov. 2019). "Matched and mismatched SOCs: A qualitative study on security operations center issues". In: *Proceedings of the ACM Conference on Computer and Communications Security*. Association for Computing Machinery, pp. 1955–1970. isbn: 9781450367479. doi: 10.1145/3319535.3354239.

Li, Nan, Yang Yu, and Zhi Hua Zhou (2012). "Diversity regularized ensemble pruning". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 7523 LNAI (PART 1), pp. 330–345. issn: 03029743. doi: 10.1007/978-3-642-33460-3_27. url: https://www.researchgate.net/publication/267404624_Diversity_Regularized_Ensemble_Pruning.

Mienye, Ibomoiye Domor and Yanxia Sun (2022). "A Survey of Ensemble Learning: Concepts, Algorithms, Applications, and Prospects". In: *IEEE Access* 10, pp. 99129–99149. issn: 21693536. doi: 10.1109/ACCESS.2022.3207287.

Mohri, Mehryar, Afshin Rostamizadeh, and Ameet Talwalkar (2012). *Foundations of Machine Learning*. isbn: 978-0-262-01825-8. url: https://www.hlevkin.com/hlevkin/45MachineDeepLearning/ML/Foundations_of_Machine_Learning.pdf.

Moradi, Mehrdad, Bert Van Acker, and Joachim Denil (Feb. 2023). "Failure Identification Using Model-Implemented Fault Injection with Domain Knowledge-Guided Reinforcement Learning". In: *Sensors 2023, Vol. 23, Page 2166* 23 (4), p. 2166. issn: 1424-8220. doi:

10.3390/S23042166. url: https://www.mdpi.com/1424-8220/23/4/2166/htm%20https://www.mdpi.com/1424-8220/23/4/2166.

Muniz, Joseph, Gary McIntyre, and Nadhem AlFardan (2015). *Security Operations Center: Building, Operating, and Maintaining your SOC*. url: https://books.google.pt/books?hl=pt-PT&lr=&id=riraCgAAQBAJ&oi=fnd&pg=PT28&ots=SBNh-G3lCL&sig=6w_ipaEbsLDSFja8OQABKEBW7GE&redir_esc=y#v=onepage&q&f=false.

Nila, Constantin, Ioana Apostol, and Victor Patriciu (June 2020). "Machine learning approach to quick incident response". In: *2020 13th International Conference on Communications, COMM 2020 - Proceedings*, pp. 291–296. doi: 10.1109/COMM48946.2020.9141989.

Rutledge, Samantha (2024). *How SOC as a Service can help Sarah in Operations*. url: https://fractionalciso.com/how-soc-as-a-service-can-help-sarah-in-operations/.

Saini, Neeraj et al. (Dec. 2023). "A hybrid ensemble machine learning model for detecting APT attacks based on network behavior anomaly detection". In: *Concurrency and Computation: Practice and Experience* 35 (28), e7865. issn: 1532-0626. doi: 10.1002/CPE.7865. url: https://researcher.manipal.edu/en/publications/a-hybrid-ensemble-machine-learning-model-for-detecting-apt-attack.

Schapire, Robert E (1990). *The Strength of Weak Learnability*.

Sethi, Kamalakanta et al. (Dec. 2020). "A context-aware robust intrusion detection system: a reinforcement learning-based approach". In: *International Journal of Information Security* 19 (6), pp. 657–678. issn: 16155270. doi: 10.1007/S10207-019-00482-7.

Shaw, Megan (Aug. 2022). *Importance of SIEM in Supporting Digital Transformation Initiatives | DNIF*. url: https://www.dnif.it/en/blog/importance-of-siem-in-supporting-digital-transformation-initiatives.

Sheeraz, Muhammad et al. (Apr. 2023). *Effective Security Monitoring Using Efficient SIEM Architecture*. doi: 10.22967/HCIS.2023.13.023.

Shirey, R (2007). *Network Working Group*. url: http://tools.ietf.org/html/rfc4949.

Sneha, N. and Tarun Gangil (Dec. 2019). "Analysis of diabetes mellitus for early prediction using optimal features selection". In: *Journal of Big Data* 6 (1). issn: 21961115. doi: 10.1186/S40537-019-0175-6.

Sopan, Awalin et al. (May 2019). "Building a Machine Learning Model for the SOC, by the Input from the SOC, and Analyzing it for the SOC". In: *2018 IEEE Symposium on Visualization for Cyber Security, VizSec 2018*. doi: 10.1109/VIZSEC.2018.8709231.

Sovilj, Dušan et al. (2020). "A comparative evaluation of unsupervised deep architectures for intrusion detection in sequential data streams". In: *Expert Systems with Applications* 159, p. 113577. issn: 0957-4174. doi: https://doi.org/10.1016/j.eswa.2020.113577. url: https://www.sciencedirect.com/science/article/pii/S0957417420304012.

Sundaramurthy, Sathya Chandran et al. (Nov. 2014). "A tale of three security operation centers". In: *Proceedings of the ACM Conference on Computer and Communications Security* 2014-November (November), pp. 43–50. issn: 15437221. doi: 10.1145/2663887.2663904. url: https://www.researchgate.net/publication/265786529_A_Tale_of_Three_Security_Operation_Centers.

Suthaharan, Shan (2016). *Machine Learning Models and Algorithms for Big Data Classification*. Vol. 36. Springer US. isbn: 978-1-4899-7640-6. doi: 10.1007/978-1-4899-7641-3. url: https://link.springer.com/10.1007/978-1-4899-7641-3.

Tao, Lin (May 2018). "A DATA TRIAGE RETRIEVAL SYSTEM FOR CYBER SECURITY OPERATIONS CENTER". In.

Tines (2023). *Voice of the SOC 2023 Report*. url: https://www.tines.com/reports/voice-of-the-soc-2023/.

Vielberth, Manfred (2021). "Security Information and Event Management (SIEM)". In: Springer Berlin Heidelberg, pp. 1–3. doi: 10.1007/978-3-642-27739-9_1681-1.

Vielberth, Manfred et al. (2020). "Security Operations Center: A Systematic Study and Open Challenges". In: *IEEE Access* 8, pp. 227756–227779. issn: 21693536. doi: 10.1109/ACCESS.2020.3045514.

Yang, Jianxin et al. (Nov. 2019). "Delineation of urban growth boundaries using a patch-based cellular automata model under multiple spatial and socio-economic scenarios". In: *Sustainability (Switzerland)* 11 (21). issn: 20711050. doi: 10.3390/SU11216159.

Zimmerman, Carson (2014). *Ten Strategies of a World-Class Cybersecurity Operations Center*. isbn: 9780692243107. url: www.mitre.org.